

Teaching philosophy: engagement and clarity

My goal is for students to build a durable understanding of the principles that govern software systems and to apply those principles with confidence. In my experience, that depth comes from genuine engagement. Therefore, I design courses around a small set of clear learning outcomes, prepare materials that connect ideas to real systems, provide opportunities to practice, and deliver feedback that treats students fairly and helps them improve. I also plan courses so they scale to different class sizes, keep feedback cycles short, and use authentic projects and capstones where they strengthen transfer from theory to practice. In particular, I focus on the five habits of practice.

High-quality, current material. I pair reliable textbooks with sources that show how the ideas live in today's systems. For example, when I built a 130-student Computer Architecture course from scratch, I used Tanenbaum as the spine and wove in manufacturer white papers on contemporary CPUs and GPUs. To reach students who learn best by building, I designed seven programming-centered labs (later published as a textbook). To keep everyone in steady contact with the material, I added twelve short quizzes spaced throughout the term; many students reported these made the midterm and final more manageable.

Communication and community. I model precise, timely communication. Students engage more readily in online technical discussion when they see consistent, substantive instructor participation. I participate consistently on course forums like Piazza so that questions get timely, precise answers and quieter students have a low-pressure way to engage. In person, I hold hands-on office hours—both scheduled and drop-in—that often become short mini-workshops.

Personalized and actionable feedback. Where automation can handle objective checks, I use it to free time for individualized, higher-level guidance and to shorten turnaround. As a TA in Compilers, I wrote an auto-checker that tested submissions so I could spend time on design comments and testing strategies rather than manual runs. As a result, technical conversations with students shifted from “does it meet the spec?” to “is this design extensible and well-tested?”—a qualitative improvement in depth.

Fair, consistent grading. In large courses, detailed individual feedback is not always possible, so I emphasize clear rubrics and calibration. As lead TA for a 150-student Software Development course, I coordinated weekly meetings to align TAs on the rubric, we anonymized grading when feasible, and we reviewed sample submissions together. Student evaluations reflected that grading felt transparent and fair.

Continuous improvement. I approach iteration as a data-driven process. Alongside brief mid-semester class check-in—an anonymous survey followed by a summary discussion—I analyze item-level results (which questions clustered errors) and maintain a teaching log of adjustments for the next offering. Going forward, I will monitor metrics such as assignment on-time completion and course completion versus fail rates, using those data to evaluate and refine changes.

Commitment to Broadening Participation

In my scholarly work, I aim for removing needless barriers so more people can do ambitious work. In a university community, it means creating norms and structures that let students, staff, and faculty with different languages, finances, and prior access participate fully.

Accessible On-ramps. I strive to make technical knowledge accessible beyond the university classroom. I have taught computing as a high-school extracurricular for several years, translating the same CS ideas I teach undergraduates into examples digestible for learners with little or no background. This experience strengthened my ability to support learners with varied math preparation. Additionally, I give brief overviews of CS subfields for sophomores; these sessions broaden their sense of what counts as CS and help them choose research projects with confidence.

Adaptability and Inclusion I adapt practices to student needs and backgrounds. For undergraduates, we scope projects to yield reproducible artifacts and poster-ready results within a term, with clear criteria for “done”. For non-native speakers or students unfamiliar with academic norms, I add written follow-ups after meetings, explicit checklists for next steps, and more frequent micro-deliverables so progress is visible. This approach was particularly effective with a French-speaking mentee I supervised in a Russian-language institution; by pacing meetings and agreeing on written summaries, we bridged the linguistic gap and completed a strong project.

Structural Advocacy. My service work aims to amplify student voices. As a co-organizer—and later president—of the Khoury Graduate Students Association at Northeastern, I helped create a safe space for students to share concerns with one another and ran anonymous surveys to surface themes we relayed to college leadership. I also contribute to PhD admissions feedback programs for applicants from historically underserved groups in CS, demystifying the tacit parts of the process—what committees look for, timelines, and framing—that are hard to infer without prior access to a large academic setting.

Courses I would like to teach

My teaching interests are largely in the languages and systems arena, but I also enjoy teaching introductory courses. I am excited to teach: CS1/CS2, Data Structures and Algorithms, Compilers and Programming Languages, Theory of Computation, and Computer Architecture.

Electives I can offer when there is demand: Optimizing and JIT Compilers, Theory of Programming Languages, Parallel and Functional Programming, and research-aligned seminars.

Teaching initiatives

Inclusive introductory programming. Students across majors increasingly need to write small programs that load, transform, and analyze data; framing introductory CS around those authentic tasks makes the course relevant beyond the CS major while still building core computing habits. I plan a refreshed intro that keeps proven pedagogy (clear weekly outcomes, frequent low-stakes practice) while updating tools and contexts in the spirit of *A Data-Centric Introduction to Computing* (DCIC). Concretely, we will use an educational language with a familiar, Python-like syntax but with more guardrails built in; introduce tabular data early; and connect control flow, data structures, and abstraction directly to these operations. We will emphasize readable code, testable functions, and reproducibility. The result is a course that lowers the barrier for non-CS students and prepares CS majors for today’s workflows in software, data science, and research computing.

AI-assisted programming. Large language models (LLMs) are now part of software practice, and I plan to teach students to use LLMs critically and ethically. My experience maintaining sizable codebases (the Cabal build system, 100K+ LOC, and the Gibbon compiler, ~50K LOC) shows where current tools fail on real tasks. Early assignments in my courses will be AI-free to build fundamentals. Later, we will introduce structured use: students must disclose assistance, write their own tests, and verify and refactor AI-generated code. The core message is simple: AI can speed exploration, but understanding remains critical.